

Hermes Runtime + Harness-Initiated Sessions

Implementation plan · canonical source preserved · 7 September 2026

Executive opening

Goal. Add Hermes as a fourth, production-capable AOS runtime and let a trusted harness create real persisted Hermes conversations that appear in AOS without fabricating ownership or browser state.

Agent model. Hermes Profiles—not personas—are isolated agents: each allowlisted Profile has its own config, skills, memory, state, credentials, and sessions. AOS maps each stable Profile slug to one immutable AOS Agent ID; it never collapses them into a fabricated default agent.

Architecture. Hermes is authoritative for sessions, transcript, runs, and cancellation. AOS resolves the selected Agent to exactly one Profile, profile-scoped API route, and server-only key; the protected inbound endpoint creates and confirms a session under that Profile, then returns its opaque ID and AOS route. The browser discovers it from the provider-owned catalog.

Locked execution instruction. Use `subagent-driven-development` task-by-task; preserve the three existing runtimes.

Stack. Next.js 16 · React 19 · Assistant UI React 0.15.18/Core 0.3.17 · Hermes API Server v0.21 · Zod · Vitest · Testing Library · Playwright.

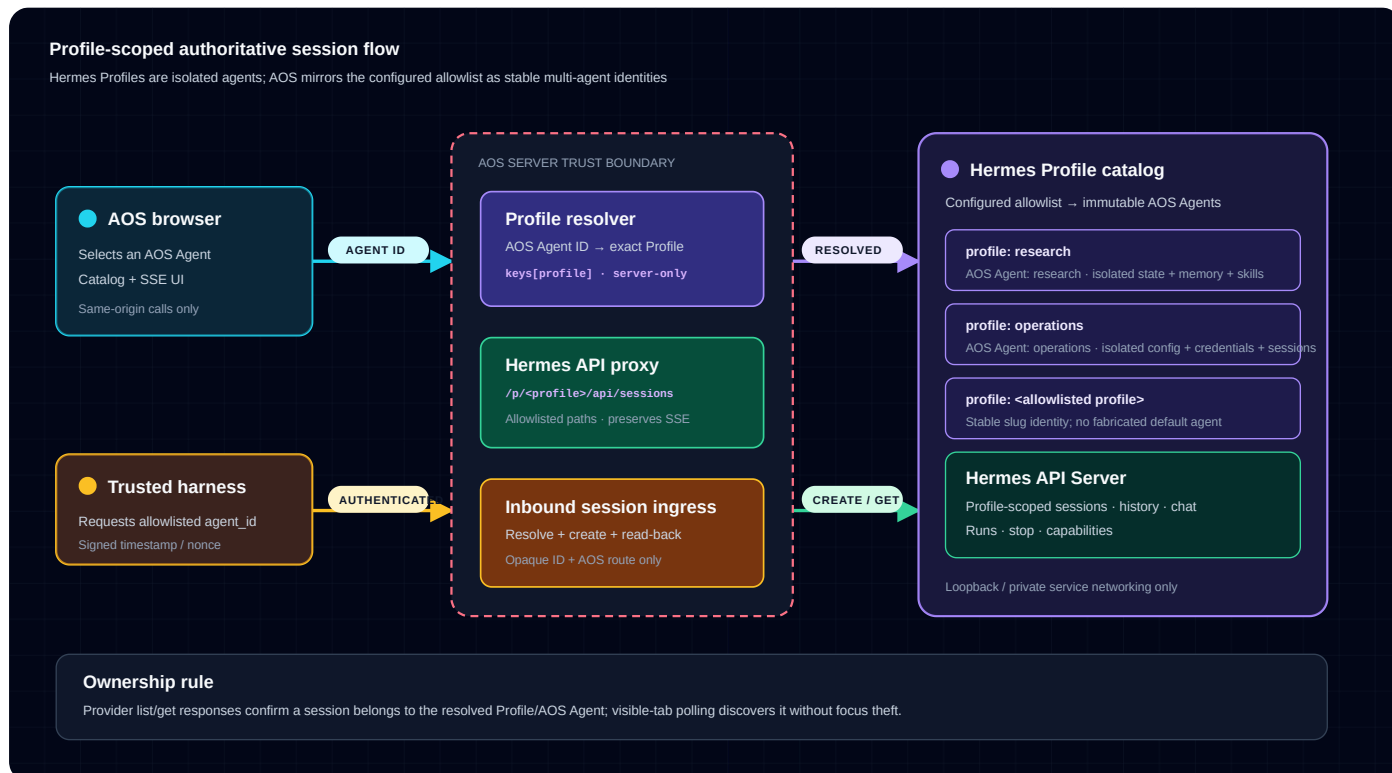


Figure 1. The authenticated ingress and proxy meet Hermes on the server; provider list/get responses are the evidence a new session belongs in AOS.

Decisions locked before implementation

- Hermes is **additive**: fixture, OpenCode, and AG-UI behavior and configuration remain unchanged.
- Hermes Profiles are AOS Agents. The configured, allowlisted profile catalog is provider-owned: each profile has isolated config, skills, memory, state, credentials, and sessions. Map every configured profile slug to one stable AOS Agent ID; do not collapse them into a fake default agent or invent client-side ownership.
- Keep `API_SERVER_KEY` server-only. Never put it in `NEXT_PUBLIC_*`, browser fetch headers, routes, query strings, logs, or SSE URLs.
- Require Hermes API capabilities for session CRUD/history, session chat streaming, run status/stop, and SSE before rendering Hermes as available.
- The currently running Hermes API listener is bound to `0.0.0.0:8642`. That is unacceptable for a terminal-capable API. Deployment must bind it to loopback/private service networking before this runtime is enabled.
- The harness receives a dedicated AOS ingress contract. It must authenticate to AOS and may request an allowlisted target Profile/Agent, title, and opaque correlation ID, but it must never provide a Hermes session ID, transcript, authorization token, or unsupported ownership claim. AOS resolves the requested Agent to its configured Hermes Profile and credentials server-side.
- Provider session list/get responses—not inbound payloads—are the proof a session exists and belongs in AOS. Inbound creation must return only after Hermes has confirmed the created session.
- The active user conversation never changes because an inbound session arrived. The new session appears under Hermes and may create a content-free Activity item; user navigation remains explicit.

Task 1: Add strict Hermes configuration and server-only proxy boundary

Files:

- Create: `lib/runtime-adapters/hermes/hermes-config.ts`
- Create: `lib/runtime-adapters/hermes/hermes-config.test.ts`
- Create: `app/api/hermes/[...path]/route.ts`
- Create: `app/api/hermes/[...path]/route.test.ts`
- Modify: `lib/runtime-config.ts`
- Modify: `lib/runtime-config.test.ts`
- Modify: `app/[...workspace]/page.tsx`
- Modify: `app/[...workspace]/page.test.tsx`
- Modify: `.env.example`
- Modify: `.env.compose.example`
- Modify: `compose.yaml`
- Modify: `README.md`

Steps:

1. Add `hermes` to `RuntimeMode` and a strict ready configuration containing a server-reachable `AOS_UI_HERMES_API_URL` plus an explicit, ordered AOS-visible Profile allowlist. Resolve profile-specific API keys server-side only; reject unknown, duplicate, or malformed profile slugs.
2. Add explicit unavailable reasons for missing/invalid Hermes URL and unavailable required capabilities. Keep invalid runtime values unavailable; never fall back to fixtures.

3. Read only the configured Profile-specific Hermes API keys in route handlers/server modules (for example, a server-only key map indexed by allowlisted Profile slug). Validate configured URL as HTTP(S); normalize it; reject userinfo, query, and fragment components.
4. Implement a same-origin Next route that proxies only an allowlist of required authenticated Hermes paths and methods. It resolves an AOS Agent ID to a configured Hermes Profile, uses that Profile's key, and routes to the default path or `/p/<profile>/...` only when multiplexing is configured: - `GET /v1/capabilities` - `GET /api/sessions`, `POST /api/sessions` - `GET/PATCH/DELETE /api/sessions/:id` - `GET /api/sessions/:id/messages` - `POST /api/sessions/:id/chat/stream` - `GET /v1/runs/:id`, `POST /v1/runs/:id/stop`
5. Preserve SSE response streaming; strip hop-by-hop headers; set no-store; do not create an open proxy or forward browser-provided Authorization headers.
6. Add isolated Hermes Compose configuration only when AOS can actually reach the API server securely. Do not expose the API port publicly. Document loopback/private-network requirements and required Hermes gateway API-server configuration.

Tests:

- Runtime config accepts only a complete Hermes configuration and leaves existing modes unchanged.
- Proxy rejects unallowlisted paths/methods and malformed IDs; adds server-side bearer auth; never echoes a secret.
- Capability handshake fails closed when required features are missing.

Task 2: Build a validated Hermes HTTP client and canonical mappers

Files:

- Create: `lib/runtime-adapters/hermes/hermes-http-transport.ts`
- Create: `lib/runtime-adapters/hermes/hermes-http-transport.test.ts`
- Create: `lib/runtime-adapters/hermes/hermes-schemas.ts`
- Create: `lib/runtime-adapters/hermes/hermes-schemas.test.ts`
- Create: `lib/runtime-adapters/hermes/hermes-message-mapper.ts`
- Create: `lib/runtime-adapters/hermes/hermes-message-mapper.test.ts`

Steps:

1. Define narrow Zod schemas for Hermes session list/create/read, message history, session-stream SSE events, run status, and capability responses. Unknown provider data stays inspectable as generic JSON; malformed required fields are rejected.
2. Implement a browser client that calls only the same-origin AOS proxy. Add pagination and stable ordering for session lists.
3. Parse `text/event-stream` incrementally. Support `run.started`, `message.started`, `assistant.delta`, `tool.progress`, `tool.started`, `tool.completed`, `tool.failed`, `assistant.completed`, `run.completed`, `error`, and `done`.
4. Map canonical Hermes messages to Assistant UI message parts. Preserve provider IDs and ordering. Map schema-compatible rich tool payloads into existing AOS renderers; all unknown or invalid tools use the existing safe generic JSON renderer.
5. Do not add a second client persistence channel: history is refreshed from Hermes and Assistant UI `append`, `update`, and `delete` are initially no-ops.

Tests:

- Zod rejects malformed IDs, messages, and stream records.
- Pagination/order is deterministic.
- Text deltas are cumulative; tool lifecycle snapshots retain stable IDs; failed tools remain visible.
- Canonical `run.completed` transcript wins after streaming/reconnect.

Task 3: Implement the Hermes Assistant UI runtime

Files:

- Create: `lib/runtime-adapters/hermes/hermes-thread-list-adapter.ts`
- Create: `lib/runtime-adapters/hermes/hermes-thread-list-adapter.test.ts`
- Create: `lib/runtime-adapters/hermes/hermes-history.ts`
- Create: `lib/runtime-adapters/hermes/hermes-history.test.ts`
- Create: `lib/runtime-adapters/hermes/hermes-chat-model.ts`
- Create: `lib/runtime-adapters/hermes/hermes-chat-model.test.ts`
- Create: `lib/runtime-adapters/hermes/use-hermes-runtime-bundle.ts`
- Create: `lib/runtime-adapters/hermes/use-hermes-runtime-bundle.test.tsx`
- Create: `lib/runtime-adapters/hermes/index.ts`

Steps:

1. Implement `RemoteThreadListAdapter` over Hermes session CRUD. Use Hermes session IDs as `remoteId`; map title and update time without deriving identity from display fields.
2. Implement `ThreadHistoryAdapter.load()` with `GET /api/sessions/:id/messages` and `ExportedMessageRepository.fromArray(...)`.
3. Use `useRemoteThreadListRuntime` with a controlled selected `threadId`, plus a per-thread `useLocalRuntime` and `ChatModelAdapter`.
4. Submit turns to `POST /api/sessions/:id/chat/stream` with a unique `Idempotency-Key`. Capture the provider `run_id` from `run.started`.
5. On Assistant UI abort, close the SSE stream and best-effort `POST /v1/runs/:id/stop`; resolve as cancelled instead of surfacing a fake transport error. On reconnect, fetch run status then canonical history.
6. Reconcile after `run.completed` by reloading authoritative history rather than trusting only client-assembled deltas.

Tests:

- Thread list, rename, delete, hydration, selection switching, and history ordering.
- Send/retry reaches the selected remote Hermes session only.
- Tool start/completed/failed UI state; text-before-tool preservation.
- Abort sends one stop request and leaves the runtime usable.

Task 4: Add a Hermes workspace adapter and honest capability surface

Files:

- Create: `lib/runtime-adapters/hermes/hermes-workspace.ts`
- Create: `lib/runtime-adapters/hermes/hermes-workspace.test.ts`
- Create: `lib/runtime-adapters/hermes/hermes-activity.ts`
- Create: `lib/runtime-adapters/hermes/hermes-activity.test.ts`
- Modify: `lib/runtime-adapters/contracts.ts`
- Modify: `lib/runtime-adapters/workspace-adapter.contract.ts`
- Modify: `components/aos-ui-workspace.tsx`
- Modify: `components/aos-ui-workspace.test.tsx`
- Modify: `lib/harness/manifests.ts`
- Modify: `lib/harness/harness-prompt.test.ts`

Steps:

1. List exactly the configured, allowlisted Hermes Profiles as immutable AOS Agents. Use stable profile slugs as IDs and Profile labels/descriptions only as display metadata; do not infer identities from titles or session fields.
2. Map `createSession(agentId, { title })` to the resolved Profile's `POST /p/<profile>/api/sessions` (or the default-profile equivalent). Reject unconfigured agents and confirm the returned session against the same Profile before caching it.
3. Add one provider-neutral optional hook to `WorkspaceAdapter`: `ts subscribeSessionCatalog?: (listener: () => void, onError?: (error: Error) => void) => () => void` It signals only that the authoritative collection changed. It carries no session payload or ownership claim.
4. In `AosUiWorkspace`, subscribe to that hook. On invalidation, reload Assistant UI threads and refresh agents; preserve the selected route/thread. Reuse existing metadata fetch and exact Profile/Agent ownership checks before rendering any new session.
5. Implement Hermes catalog invalidation with visible-tab polling/reconciliation first; there is no documented global Hermes session event stream. Use a single bounded interval with visibility pause/resume and overlap prevention. Do not add a new realtime service.
6. Publish content-free run and attention Activity only after provider metadata confirms exact thread-to-agent ownership. If Hermes version/capabilities cannot surface todos, permissions, or subagents safely, advertise them as unavailable—do not simulate OpenCode capabilities.
7. Add a Hermes-specific presentation manifest that declares only verified Hermes capabilities. Do not reuse the OpenCode plugin or claim native UI tools that Hermes does not expose.

Tests:

- Extend the shared workspace adapter contract.
- Verify that the catalog includes only configured Profiles, sessions never cross profile boundaries, and an AOS Agent ID is resolved to one exact Hermes Profile on list, create, hydrate, send, retry, activity, and navigation.
- Inbound catalog invalidation reloads the provider list; unknown payload data never inserts a session.
- New sessions retain current focus and appear only after Hermes list/get confirmation.
- Activity revalidates ownership before ingest/open; deleted sessions remain unavailable.
- Polling pauses when hidden and deduplicates replays/reconnects.

Task 5: Create the trusted harness ingress for new conversations

Files:

- Create: `app/api/hermes/inbound-sessions/route.ts`

- Create: `app/api/hermes/inbound-sessions/route.test.ts`
- Create: `lib/runtime-adapters/hermes/inbound-session.ts`
- Create: `lib/runtime-adapters/hermes/inbound-session.test.ts`
- Modify: `.env.example`
- Modify: `.env.compose.example`
- Modify: `README.md`
- Modify: `docs/` (new `docs/hermes-inbound-sessions.md`)

Steps:

1. Define a small versioned request contract for the harness: `json { "agent_id": "allowlisted Hermes Profile/AOS Agent", "title": "optional display title", "correlation_id": "opaque optional id" }`
Reject unknown fields, oversized strings, duplicate correlation IDs, caller-supplied Hermes/AOS session IDs, unconfigured `agent_id`, messages, model overrides, and all secret-bearing fields.
2. Require a dedicated `AOS_HARNESS_INGRESS_SECRET` using constant-time comparison and a signed timestamp/nonce replay window. This secret authenticates only this ingress; it is not the Hermes bearer token.
3. Server-side, resolve `agent_id` to its configured Hermes Profile and create the Hermes session with that Profile's internal transport. Include the AOS Hermes presentation guidance only as layered system prompt/instructions if the documented Hermes endpoint supports it, then read it back through the same Profile. Treat returned `session.id` as authoritative.
4. Persist deduplication/correlation state only if it is already available in the deployment substrate. Otherwise require a unique one-shot idempotency key and return `409` on an exact replay. Do not introduce a database just for this feature.
5. Return `201` with opaque identifiers and a validated relative AOS path (`/<agent-id>/<session-id>`), never Hermes credentials or raw provider responses.
6. The browser discovers the new session through the catalog invalidation polling from Task 4. Do not use a harness event to auto-select it; show the existing normal session list/Activity behavior after provider confirmation.

Tests:

- Valid request creates exactly one Hermes session and returns the provider ID/path.
- Replays are idempotent; malformed, stale, unsigned, and oversized requests are rejected.
- Ingress accepts only an allowlisted Agent/Profile selector, resolves it server-side, rejects all ownership/session assertions, and never exposes bearer tokens.
- Read-after-create failure returns an explicit error and does not report a usable AOS conversation.

Task 6: Compose, harden, and verify the runtime

Files:

- Create: `components/aos-ui/hermes-app.tsx`
- Create: `components/aos-ui/hermes-app.test.tsx`
- Modify: `app/[...workspace]/page.tsx`
- Modify: `app/[...workspace]/page.test.tsx`
- Modify: `components/runtime-unavailable.tsx`
- Modify: `components/runtime-unavailable.test.tsx`
- Modify: `README.md`

- Modify: `e2e/support/provider-mock.ts`
- Create: `e2e/hermes.runtime.spec.ts`
- Modify: `package.json` and/or add a focused Playwright config only if the project's existing test architecture requires it.

Steps:

1. Add `HermesAosUiApp` as a thin provider composition seam and dynamically select it only for the resolved `hermes` mode.
2. Add localized unavailable/error copy in English and Hebrew; keep directionality, keyboard behavior, reduced motion, and visual design lock intact.
3. Extend the provider mock with session CRUD, message hydration, session-stream SSE, stop, and inbound-session flows. Do not call a real Hermes agent in normal CI.
4. Run the project's static/adaptor/component checks, then fixture/OpenCode/AG-UI regression suites to prove this is additive.
5. Run Hermes-focused Playwright journeys: initial hydration, create session, streamed tool turn, cancellation, inbound harness session appears without focus theft, replay/deduplication, and Hebrew RTL routing.
6. Before a live smoke, verify `GET /v1/capabilities` using the effective API key and prove API bind/CORS are restricted. The current listener/key state must be fixed first; do not fake this verification.

Validation commands:

```
bun run test
bun run typecheck
bun run lint
bun run build
bun run test:e2e
```

For the final opt-in smoke only, after safe API configuration:

```
curl -fsS -H "Authorization: Bearer $HERMES_API_SERVER_KEY" \
  http://127.0.0.1:8642/v1/capabilities
```

Risks and non-goals

- No browser-direct Hermes access, no permissive CORS, no external API exposure, and no harness access to terminal-capable Hermes credentials.
- No generic Hermes webhook integration for session creation.
- No fake Hermes agent catalog, todos, permissions, subagent UI, rich tool capability, or client-owned transcript persistence.
- No new realtime infrastructure. Bounded visible-tab catalog polling is enough to make inbound sessions discoverable; reassess only after there is evidence it blocks use.
- If live Hermes capability/version differs from the documented contract, render unavailable rather than silently downgrading to fixtures.